Khuzdoor Thesis

Zevi Berlin et al.

March 3, 2026

Introduction

Khuzdoor, named after the word for "Dwarf" in the Dwarvish language in *Lord of the Rings*, "Khuzûd" and the word "door" – the name comes from scene #33 in the extended cut, #27 in the theatrical cut of *The Fellowship of the Ring* (2001) where The Fellowship enters Moria, the secret dwarf city. In order to enter, they must find a door, which is perfectly camouflaged in its surroundings, and speak the password. Khuzdoor as an app hides data, which is encrypted using a password-derived AEGIS-256 key, inside image, video, or audio files.[1]

Theorem

"The ultimate encryption is where an adversary can't even tell you have data"

Generally, the accepted "best way" for Alice and Bob to privately communicate is to use some form of Hybrid Authenticated Encryption (HAE). However, the system still has the flaw that attackers can easily see that Alice and Bob are communicating. This could, for instance, lead to an attacker using a Harvest Now Decrypt Later (HNDL) attack. The simplest solution for this problem of secrecy is steganography, which is the process of hiding secret data within a large pool of analog data, such as an image.

This paper lists all mechanisms, attacks, libraries, and systems it uses; then in the *Res Ipse* section it actually pulls everything together and describes the program map and workflow.

[1] Images are currently the only filetype supported, both video and audio are coming soon with FFmpeg

"stego data", "stego image", etc: host data that has been altered, using steganography, to hide secret data.

Steganographic Techniques

Appendation Steganography

With standard steganography, secret data is simply appended to the end of the host data, with almost no effort made to camouflage it whatsoever. When a computer attempts to read the host data, such as to render an image or play a video, it *usually* ignores the added secret data. Therefore, without inspecting the source data, which can present challenges to non-technical people, an unsuspecting person would have absolutely no idea of the secret data's presence. However, upon a quick reveal of the source, one could easily spot the plaintext data at the very end. With the original data, the size of the stego data clearly shows discrepancy that immediately points to added data. Anybody with a tool that reads data could clearly extract the added data as well. While appendation steganography works to conceal data, its safety is almost null and its privacy is negligible. It *should not* be used for any purpose other than grouping data together. Khuzdoor does **not** use this technique, ever.

LSB Steganography

Least Significant Bit (LSB) steganography is a method of changing the least significant bit of pixels within an image to encode the secret data within the host data.

For example:

The character 'P' can be encoded based on an ascii table into binary:

$$\text{'P'} \rightarrow 01010000$$

In binary, the LSB is the final one to the right, so with the host data:

$$01100100\ 01100001\ 01110100\ 01100001\ 00100000\ 01101111$$
$$00100000\ 01100101\ 01101001\ 01100111\ 01101000\ 01110100$$

The LSBs can be changed such that they match the bits of the character:

$$\text{To alter the byte `10111001` to match the bit `0`, the LSB changes}$$
$$1011100\mathbf{1} \rightarrow 1011100\mathbf{0}$$

$$\text{Following suit for the host data}$$
$$0110010\mathbf{0} \mid 0 \rightarrow 0110010\mathbf{0}$$
$$0110000\mathbf{1} \mid 1 \rightarrow 0110000\mathbf{1}$$
$$0110000\mathbf{1} \mid 0 \rightarrow 0110000\mathbf{0}$$
$$0111010\mathbf{0} \mid 1 \rightarrow 0111010\mathbf{1}$$
$$0010000\mathbf{0} \mid 0 \rightarrow 0010000\mathbf{0}$$
$$0110111\mathbf{1} \mid 0 \rightarrow 0110111\mathbf{0}$$
$$0010000\mathbf{0} \mid 0 \rightarrow 0010000\mathbf{0}$$
$$0110010\mathbf{1} \mid 0 \rightarrow 0110010\mathbf{0}$$
$$01101001 \rightarrow \textit{unchanged}$$
$$01100111 \rightarrow \textit{unchanged}$$
$$01101000 \rightarrow \textit{unchanged}$$
$$01110100 \rightarrow \textit{unchanged}$$

While this change may seem obvious when looking at a raw byte string, if the bytes represent pixel RGB color values, the change to the human eye is imperceptible under most circumstances. In fact, in order to determine with 100% certainty whether or not the image is a stego image, an original copy of the unchanged host image would need to be used in a pixel-for-pixel verification.

However, due to nature and camera sensors, the LSBs of naturally taken (rather than digitally created) images have a statistical randomness that is generally non-uniform and modelable. When this unique randomness pattern which occurs naturally is altered via LSB encoding, it generally becomes more uniform in areas of the spatial domain of the image.[2] "Too-Perfect" randomness is an immediate red flag for statistical steganalysis, especially because the next-to LSB remains unchanged, so the original non-uniform randomness is preserved in it; these facts can be mitigated using the technique discussed below.

LSBMR Steganography

Standing for Least Significant Bit Matching Revisited. Just like LSB steganography: it relies on changing the LSB of a pixel in order to encode a secret bit. However, this takes a different approach in that if the LSB doesn't match the secret bit, it randomly increments or decrements it which prevents the uniform randomness distribution seen above. That is the primary advantage of LSBMR over plain LSB.

[2] Wayner, Peter (2009). *Disappearing Cryptography: Information Hiding: Steganography & Watermarking*, Morgan Kaufmann Publishers, Amsterdam; Boston

A simple implementation in C++ might look like:

```cpp
#include <cstdint>

#include <sodium.h>


void writeLSBMR(uint8_t* pixel, bool bit) {
  if ((*pixel & 1U) == (uint8_t)bit) {  // pixel's LSB already is bit
      return;                           // no change necessary
    }

    // pseudo-random way to determine +/-
    if (randombytes_random() % 2 == 0) {
      *pixel = (*pixel == 255) ? 254 : (*pixel) - 1;  // avoid overflow
    } else {
      *pixel = (*pixel == 0) ? 1 : (*pixel) + 1;  // avoid underflow
    }
}
```

Or in Python:

```python
from nacl.utils import random
from nacl.bindings import randombytes_random

def writeLSBMR(pixel: bytearray, bit: bool) -> None:
    if (pixel[0] & 1) == bit:
        return

    if randombytes_random() % 2 == 0:
        pixel[0] = 254 if pixel[0] == 255 else pixel[0] - 1
    else:
        pixel[0] = 1 if pixel[0] == 0 else pixel[0] + 1
```

Attack Vectors

Chi-Square Attack (χ^2)

This form of statistical analysis is a very simple method of detecting sequential LSB steganography, but has weaknesses when it comes to random, edge, or texture steganography (all of which are discussed in great detail below). χ^2 measures frequencies on finite areas of an image, and based on the frequency distribution can find statistically significant anomalies. In the context of steganalysis, the attack measures LSBs of pixels and based on the frequency of LSBs, decides whether or not there exists anomalous data (which might flag any encoded secret bits). However, the test has a major weakness in that with large sample sizes, even tiny inconsistencies cause huge spikes, so with large images the test can be unreliable in that it generates lots of false positives.[3]

Regular-Singular Attack

RS for short, utilizing masks for small pixel regions to mimic what LSB encoding or decoding might look like and finding differences allows the algorithm to classify a ratio of "regular" pixel groups, in which flipping LSBs creates more noise (expected in natural images), vs. "singular" pixel groups, with the opposite effect. This has proven to be a *highly* effective technique for detecting sequential steganography and (sometimes) random steganography. Fortunately, or unfortunately (depending on which side of this program you're on), RS has a weakness for groups that are noisy by default, so edge and texture steganography can evade the analysis consistently if implemented correctly.

[3] Wikipedia Contributors. "Chi-Squared Test." *Wikipedia*, Wikimedia Foundation, 3 Apr. 2019, en.wikipedia.org/wiki/Chi-squared_test.

Edge and Texture Detection

Sobel Operator

A simple and efficient way to detect both weak and strong edges in images, Sobel uses a filter matrix on grayscale pixels to find a gradient, which it then compares to a set edge threshold.

First, an RGB image must be converted to grayscale. A simple enough method is to just take the average of the colors, however this ends up not working particularly well because the human perceives red, green, and blue with different luminosities. Instead, use simple conversion coefficients; different libraries use different numbers, Khuzdoor uses the ones from this:

$$\text{Luminosity of Pixel in the Grayscale}$$
$$(R, G, B) \rightarrow (0.299 * R) + (0.587 * G) + (0.114 * B)$$

With this method, grayscale images are created that look like:

Original RGB image

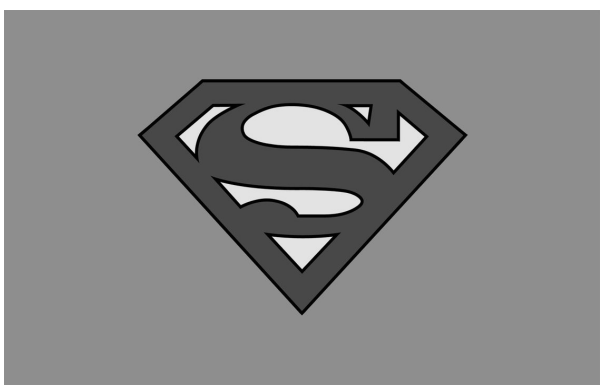


Image converted to grayscale

A grayscale image is important because Sobel operates on luminosities rather than color to detect edge gradients. Using this grayscale image with pixel luminosity values, iterate through

the image pixels starting at $(1,\ 1)$ and going to $(l\ -\ 1,\ w\ -\ 1)$ where $l,\ w$ are the length and width of the image. This iteration is so that a 3x3 matrix of pixels can always be selected and Sobel can be applied to it. The Sobel operators are as follows:

$$\text{Vertical Operator} \qquad \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \qquad\qquad \text{Horizontal Operator} \qquad \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

To use these, take your 3x3 matrix of grayscale pixels and multiply their luminosities by the respective coefficient in the operator and sum the results:

$$\begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * \begin{bmatrix} 212 & 202 & 206 \\ 190 & 184 & 181 \\ 195 & 189 & 190 \end{bmatrix} = G_y = -29$$

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * \begin{bmatrix} 212 & 202 & 206 \\ 190 & 184 & 181 \\ 195 & 189 & 190 \end{bmatrix} = G_x = -59$$

With these new directional gradients, we can determine different forms of information:

$$G_y = -29$$
$$G_x = -59$$

$$G = \sqrt{G_x^2 + G_y^2} \simeq 66$$

if $G >= threshold$ then the pixel matrix is an edge, the Khuzdoor threshold is 150

a faster but less accurate way to check the gradient is with

$$G = |G_x + G_y| = 88$$

the angle of the edge is also calculable (after it is determined to be an edge) with

$$\theta = \operatorname{atan} 2(G_y, G_x) =$$

**this is not to be confused with the $\arctan^2$ operation

Khuzdoor by default uses the absolute value method, however using the `*--strict*` flag when running the program forces it to be strict with choosing its edges by using a slightly higher threshold and the slower but more accurate Pythagorean Addition method for calculating gradient. It also affects the texture detection but this paper expands on that further below.

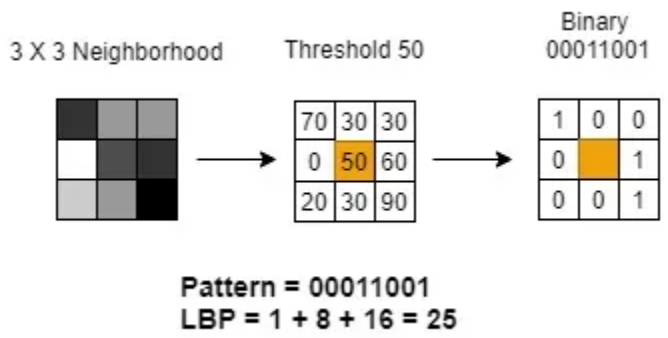
Local Binary Pattern Operator

LBP is a simple Sobel-esque method for detecting texture within an image. It uses a grayscale image (talked about above in Sobel) and a 3x3 pixel matrix to compare 8 border pixels to a central pixel. Iterate through the image pixels starting at $(1,\ 1)$ and going to $(l - 1,\ w - 1)$ where $l,\ w$ are the length and width of the image. This iteration is so that a 3x3 matrix of pixels can always be selected and Sobel can be applied to it.

First, the threshold for the cell is picked as the luminosity of the central pixel. Then, all neighboring pixels are marked as either 0 or 1 based on its luminosity compared to the threshold.

With all neighboring pixels marked, they are turned into an 8-bit binary number starting at the top-left corner and from the LSB. This number is compared to the preset texture threshold (not to be confused with the central pixel of the cell) and classified as a texture area or not.

The LBP operators act as follows:



Visual representation, Medium.com

This method has many advantages, mainly involving its computational efficiency and ease of implementation. It can also be used instead to find relative texture by first adding all scores for all cells throughout an image to a feature vector, then finding through various methods a "mean minimum" texture threshold to find texture in an image based on the image itself, rather than an arbitrary threshold. This, of course, breaks apart when it comes to *most* digitally created images because they retain little to no artifacts or natural texture.

As seen above, the `--*strict*` flag forces the program to pick a lower texture threshold. The `--*relative*` flag makes the program find relative texture within an image. However, it's important to note that the `--*relative*` flag is only a strongly-worded suggestion to the program, if it cannot find adequate texture it ignores the flag and uses other methods of steganography.

Encryption

AEGIS-256

Khuzdoor uses AEGIS-256 instead of AES-256 because AEGIS is authenticated by default, safer to implement, included in Libsodium, and can be optimized on processors without special hardware. In the future, after initial release, *The Phi Project* authors may consider adding an option to select AES-256 rather than AEGIS-256 through command-line arguments.

HMAC-based Key Derivation Function

Khuzdoor needs to be able to extract symmetric keys, nonces, and random seeds from just a single given password. For this, it needs to be able to use an HKDF. These mechanisms can take in any data, and deterministically expand it into a fixed-length pseudorandom key which can be extended further arbitrarily into any type of key or nonce. The program uses Libsodium's implementation of HKDF to extract first an AEGIS-256 key and then a corresponding nonce. In this way, a password can always be used for both encryption and decryption.

Res Ipse

Encryption Pipeline

Khuzdoor takes in an image, a password, and an input file. First, it derives an encryption key and nonce from the password. It then compresses the data using Zlib's GZIP level 5 and encrypts the compressed data using AEGIS-256 and the derived credentials. After it processes all of the data and loads the image, it does simple image format and type checks to ensure that the image being used is safe to use and compatible; it also does simple bounds checks. Next, it derives and analyzes the statistical properties of the image regarding edges and texture. It uses the Sobel operators to find any edge pixels (or just strong edges based on the presence of the `*--strict*` flag), and the LBP operator to either make a feature map or find textured area pixels (based on the presence of the `*--relative*` flag).

In the best case scenario, the host image will have enough edge pixels to hold the secret data that the app can simply encode into the edges using LSBMR. Unfortunately, some images do not have enough edge pixels, so the next best option is texture encoding. If the host image has enough texture pixels, then Khuzdoor encodes the secret data into them with LSBMR.

If both still fail, then the program will use the password to seed a deterministic random pool which provides data to shuffle a list of indices using the Fisher-Yates algorithm such that with the same password (and same image) the final list of random indices is always the same. Using these pixel indices, the program encodes data using LSBMR where the bits can be spread out in order to mitigate detection as best it can.

For decryption, follow the same steps except whenever something is encoded, decode it instead. Remember that everything is deterministic based on the password and image.